Synchronization Markers for DNAFormer: Stage 4 Results

FYP, DNA data storage · 25 October 2026

Spacing synchronization markers through the strand raised fine-tuned DNAFormer's exact recovery at 8% deletion from **41.0%** to **52.4%**, against a control with identical payload and marker content.

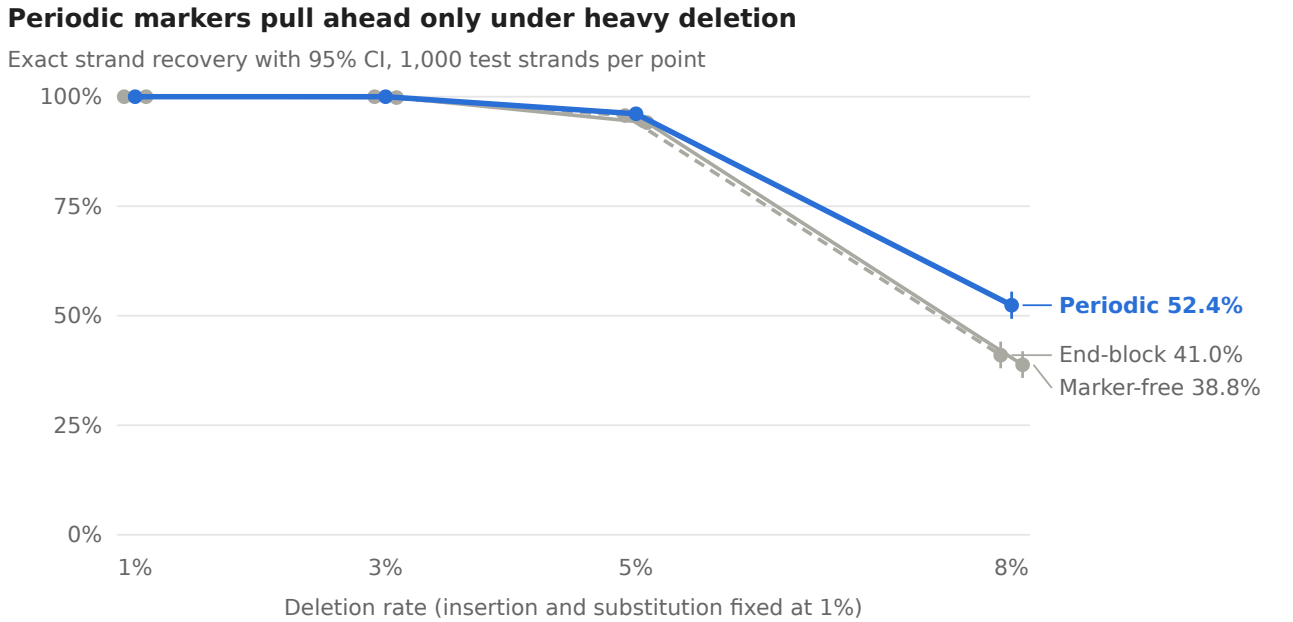
Question and setup

Do periodic markers (`ACGTCAGT` after every 24 data bases) help a fine-tuned DNAFormer recover 128-base strands when deletions are frequent?

Version	Layout of the 128 bases	Data bases
Marker-free	128 data	128
Periodic	4 × (24 data + 8 marker)	96
End-block (control)	96 data, then the 32 marker bases together	96

- **Data:** DNAFormer's own simulator; 10 noisy reads per strand; insertions and substitutions fixed at 1%; deletions at 1, 3, 5 and 8%. The versions share the same payloads and noise seeds.
- **Training:** each version fine-tuned from the same pretrained checkpoint with identical settings (10 epochs × 625 steps, best epoch by validation).
- **Scoring:** 1,000 unseen test strands per deletion level; markers removed, only data bases scored.

Results



Version	Data bases	1%	3%	5%	8%	Errors at 8% (NED)	Data recovered at 8%

							(bases per 1,000 strands)
Marker-free	128	100%	99.8%	94.1%	38.8%	0.020	49,664
Periodic markers	96	100%	100%	96.1%	**52.4%**	**0.015**	**50,304**
End-block control	96	100%	100%	95.7%	41.0%	0.024	39,360

Exact strand recovery of fine-tuned DNAFormer by deletion rate; 1,000 unseen test strands per level; markers removed before scoring.

Findings

1. **Marker spacing drives the gain.** Periodic and end-block carry the same 96 data and 32 marker bases; only placement differs. At 8% deletion periodic recovers 52.4% [49.3, 55.5] vs 41.0% [38.0, 44.1]: the intervals do not overlap.
2. **A shorter payload alone barely helps.** End-block vs marker-free at 8% is 41.0% vs 38.8% (overlapping intervals), and its edit distance is worse (0.024 vs 0.020).
3. **The effect appears only under heavy deletion.** All versions are near-perfect at 1–3%; at 5% they sit within 94–96% with overlapping intervals.
4. **Markers just pay for their 25% overhead.** At 8%, periodic recovers 50,304 data bases per 1,000 strands vs 49,664 marker-free (+1.3%).
5. **Fine-tuning itself helps every version.** At 8%, exact recovery rose from about 8% (pretrained) to 39–52%.

Is it a worthwhile finding?

Yes, for an FYP: it is a clean, controlled answer with an honest catch.

- **Why it counts:** the end-block control isolates synchronization from the shorter payload, so the claim is causal, not just "markers scored higher". It shows a classic coding idea (watermark-style markers, Davey and MacKay 2001) still helps a strong learned decoder.
- **The catch:** on stored data per strand the markers only break even (+1.3% at 8%), so they are not yet a practical win.
- **Limits:** simulated noise only; one marker sequence, one spacing (24) and one strand length; one training run per version; 10,000 training strands.
- **Before final claims:** Stage 5 adds paired tests on the same strands across versions and the final graphs. Worth testing next: other marker spacings or shorter markers, which could turn break-even into a net gain.

How we fine-tuned DNAformer

We kept DNAformer exactly as it was and only adjusted its existing weights, in two steps: first the CNN layers alone, then the whole model.

The model and what we trained

Trained in step 1 and step 2 | Frozen in step 1, trained in step 2

Input: 10 noisy reads of one strand (plus the same reads reversed)

▼

1. Alignment block — 8 convolution layers + 3 linear layers. Looks at each read and lines it up. **CNN**

▼

2. Embedding block — 8 convolution layers + 3 linear layers. Combines the 10 reads into one summary. **CNN**

▼

3. Transformer — 12 layers (about 100.8M of the 103.4M parameters). Uses the whole strand to decide each letter. **Transformer**

▼

4. Output + combine blocks — 3 + 3 small convolution layers. Turns the result into letters and merges the forward and reversed views. **CNN**

▼

Output: the 128-letter strand

The two training steps

	Step 1 (epochs 1–2)	**Step 2 (epochs 3–10)**
What learns	Only the CNN parts: all 22 CNN layers + 6 linear layers (2.6M parameters)	Everything: CNN parts + all 12 transformer layers (103.4M parameters)
What is frozen	All 12 transformer layers	Nothing
Why	Teach the “reading” layers to handle many deletions and the markers, without disturbing what the transformer already knows	Let the whole model adjust together, gently
Size of each update (learning rate)	Small and steady (0.00001)	Starts very small, rises a little (to 0.000003), then slowly shrinks to almost zero (0.0000001)

The learning rate is how big each correction is. It is tiny because the model is already trained: we only nudge it, and the corrections get smaller as it settles.

Other settings (same for all three versions)

Training data	10,000 strands, deletion 1/3/5/8% mixed	**Batch**	16 strands per update
Training length	10 passes over the data (6,250 updates)	**Seed**	1 (makes runs repeatable)
Optimizer and loss	Adam; DNAformer's own loss	**GPU / time**	Colab T4, ~35–40 min
Best model	The epoch with the lowest error on 1,000 validation strands	**Final test**	1,000 new strands per deletion level, used once